



CS 498: Batch Serving Techniques

Fan Lai

The Grainger College of Engineering

Materials with thanks to Minjia Zhang, Arvind Krishnamurthy,
and many colleagues

LLM Serving

- Basics (Prefilling and Decoding)
- Continuous Batching
- Chunked-Prefill

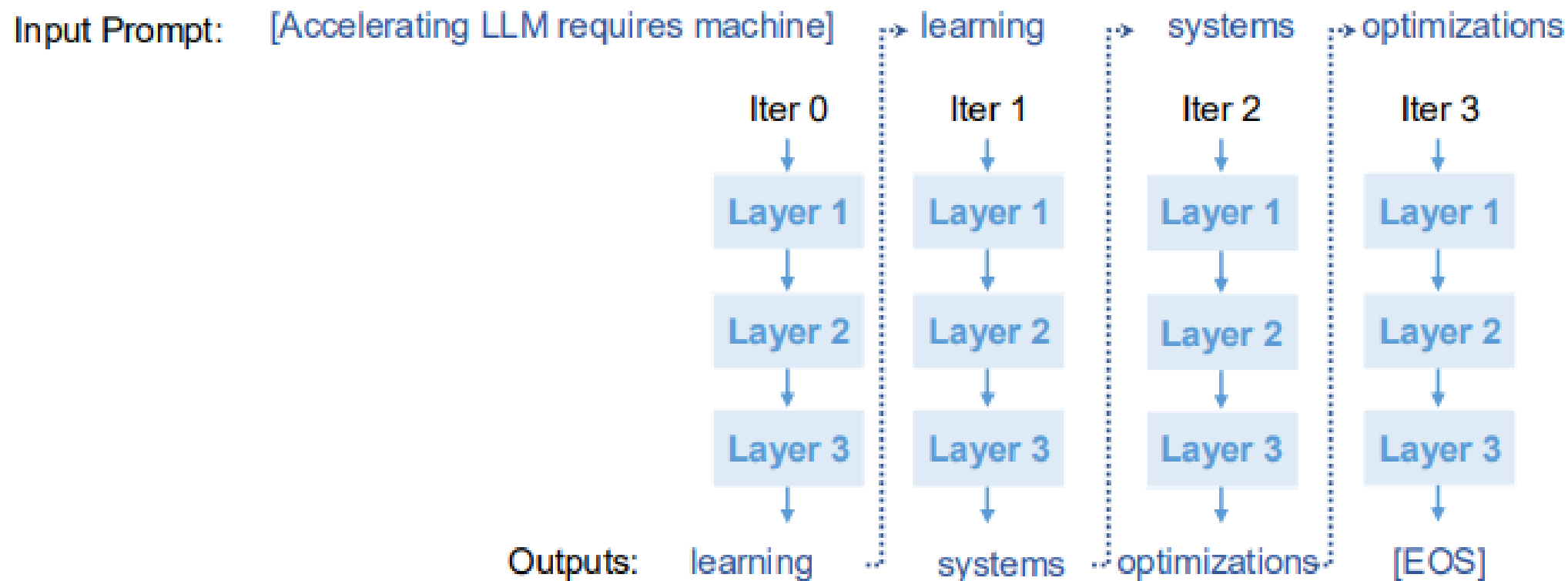
Panel Discussion (DistServe)

LLMs are Slow and Expensive to Serve

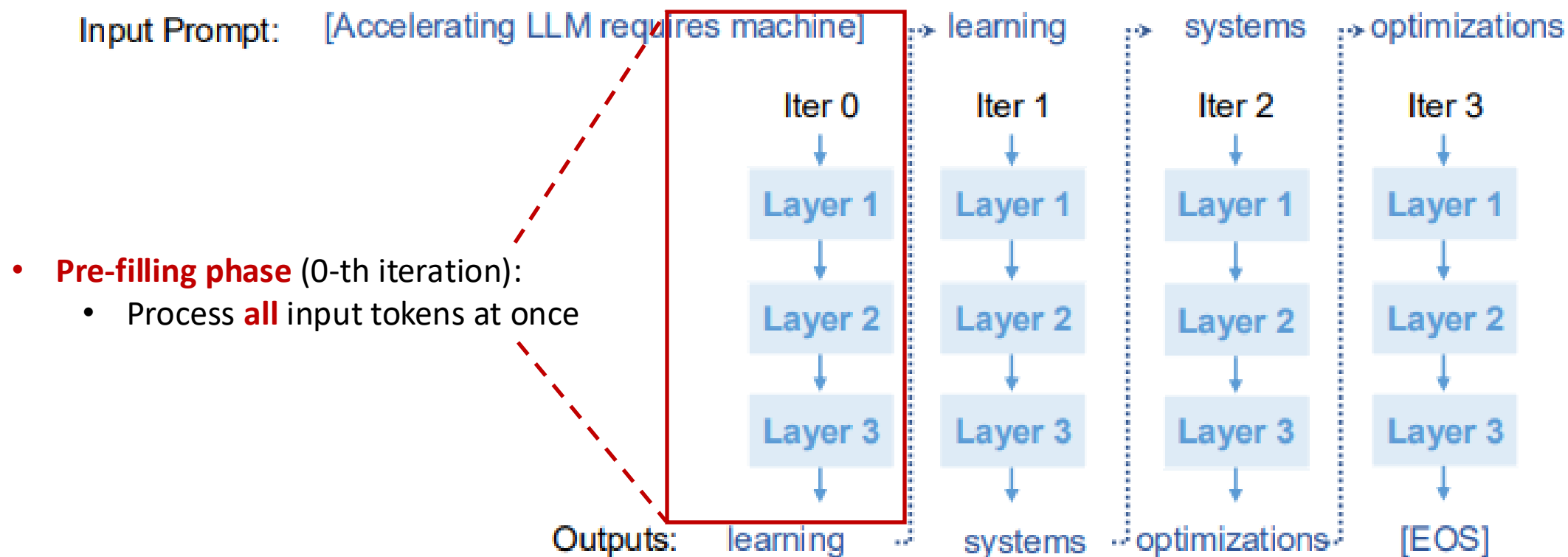


- At least ten A100-40GB GPUs to serve 175B GPT-3 in half-precision
- Generating 256 tokens takes ~20 seconds
- Cannot process requests in parallel
 - Per-request key-value cache takes 3GB GPU memory

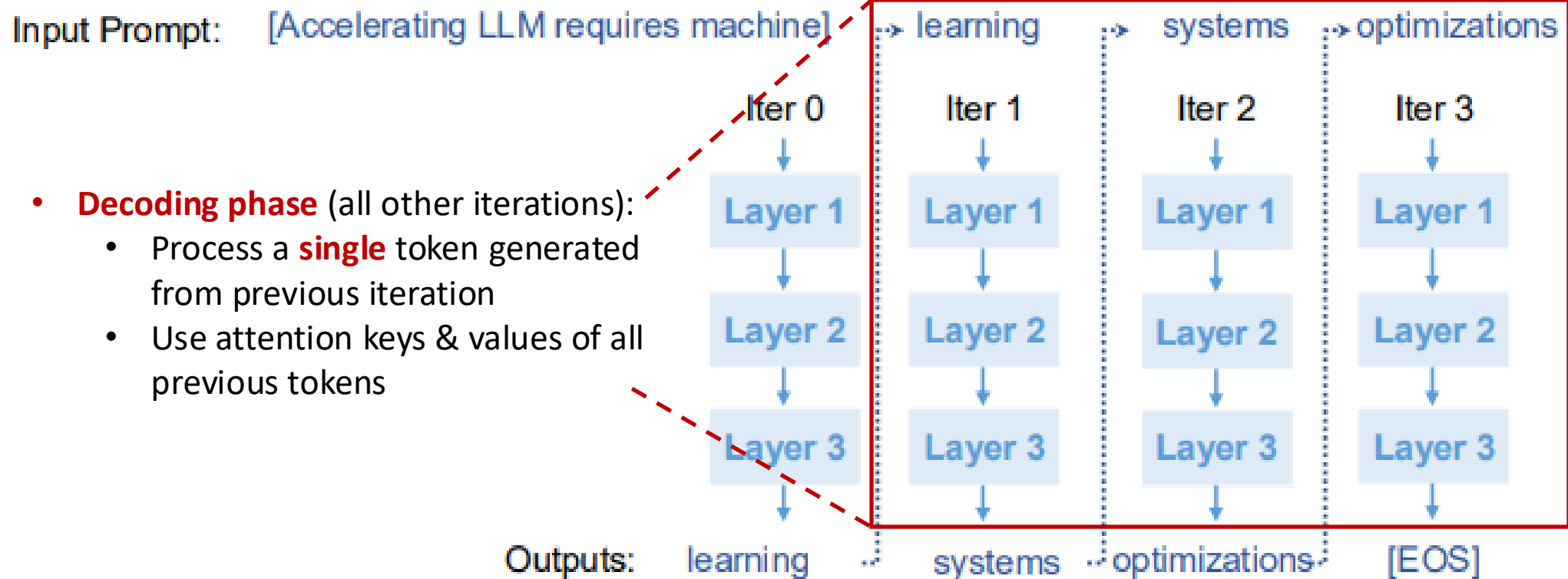
Generative LLM Inference: Autoregressive Decoding



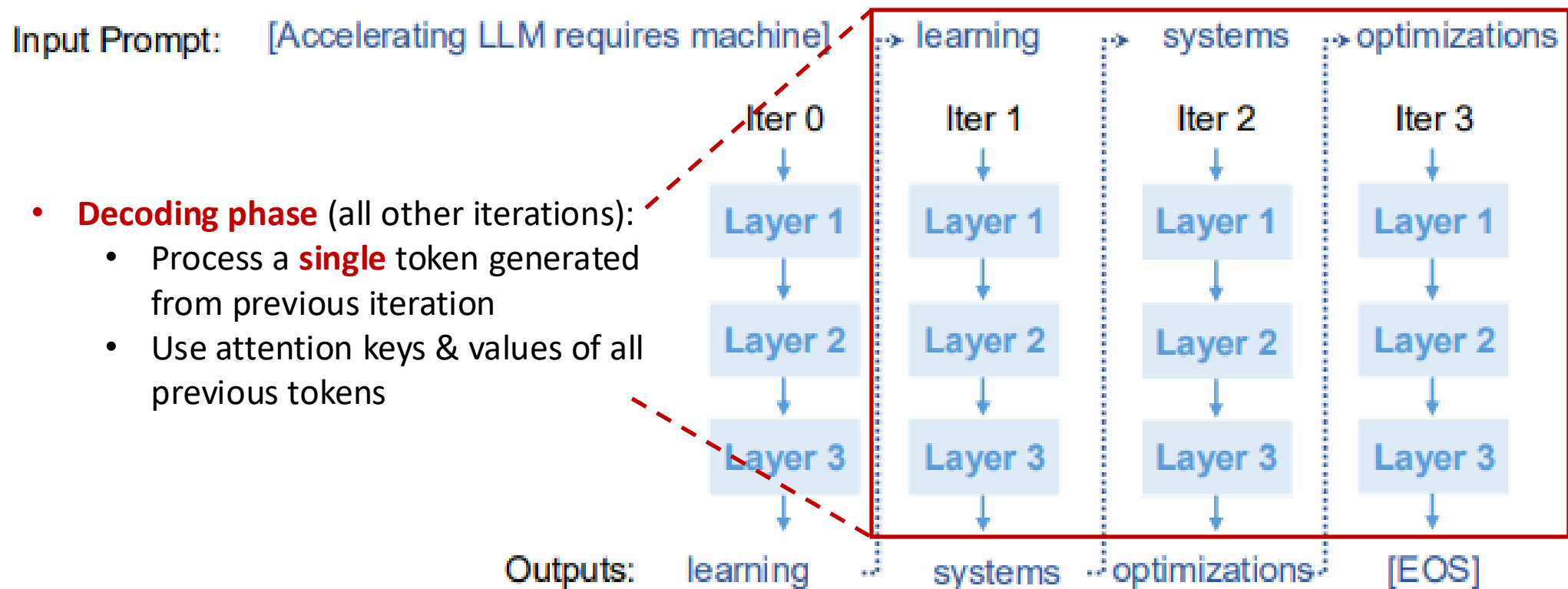
Generative LLM Inference: Autoregressive Decoding



Generative LLM Inference: Autoregressive Decoding



Generative LLM Inference: Autoregressive Decoding



Repeat until the sequence

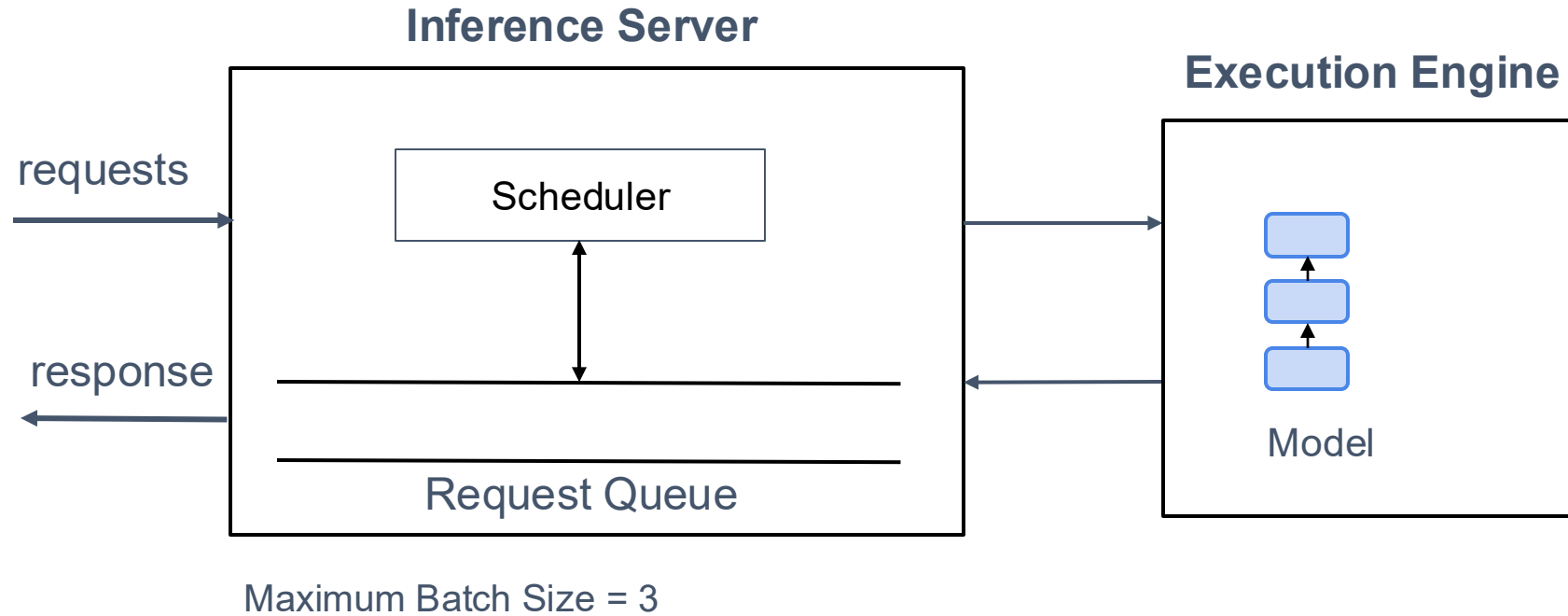
- Reaches the pre-defined maximum length (e.g., 2048 tokens)
- Generates the stop tokens (e.g., "<|end of sequence|>")

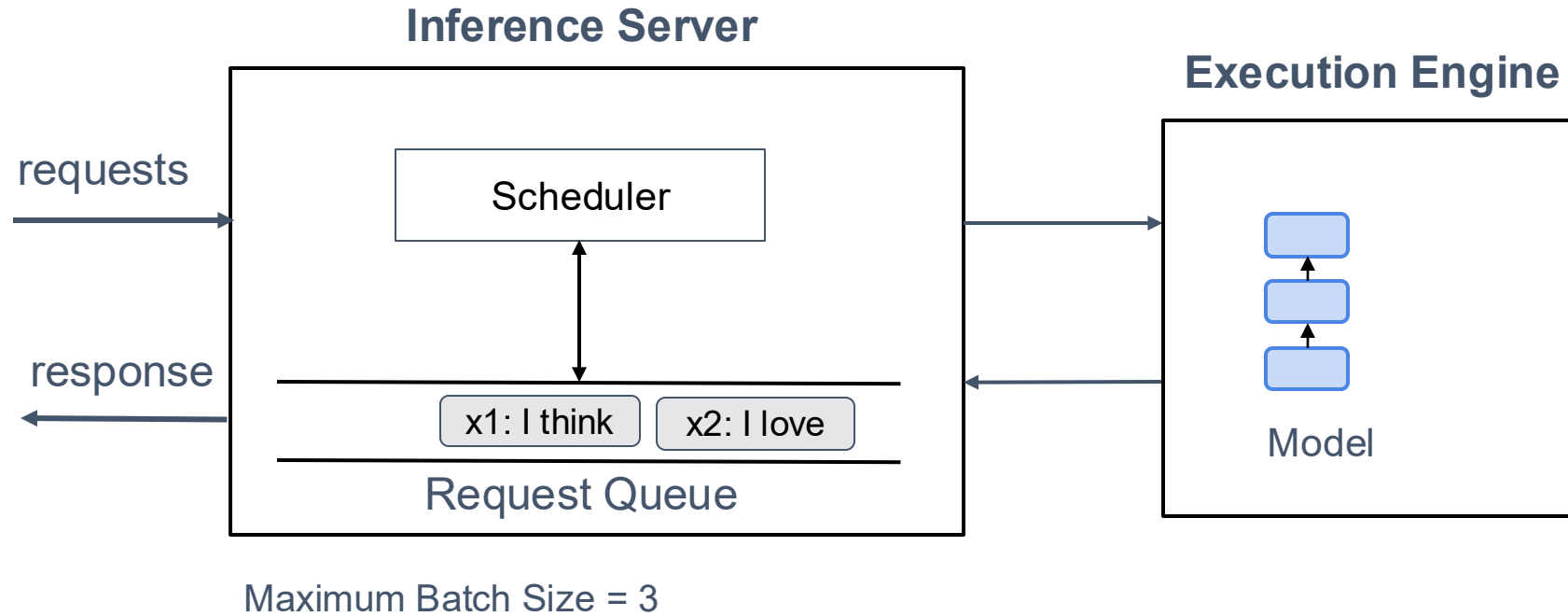
- **Time to First Token (TTFT):** Measures how quickly users begin to see the model output token after submitting a query.
 - Critical for real-time interactions
 - Driven by prompt processing time and the generation of the first token
- **Time per Output Token (TPOT):** Time taken to generate each output token.
 - Impacts user perception of speed (e.g., 100ms/token = 10 tokens/second)
- **E2E Latency = TTFT + (TPOT * the number of generated tokens)**
 - Total time to generate the complete response
- **Throughput:** Number of tokens generated per second across all requests by the inference server

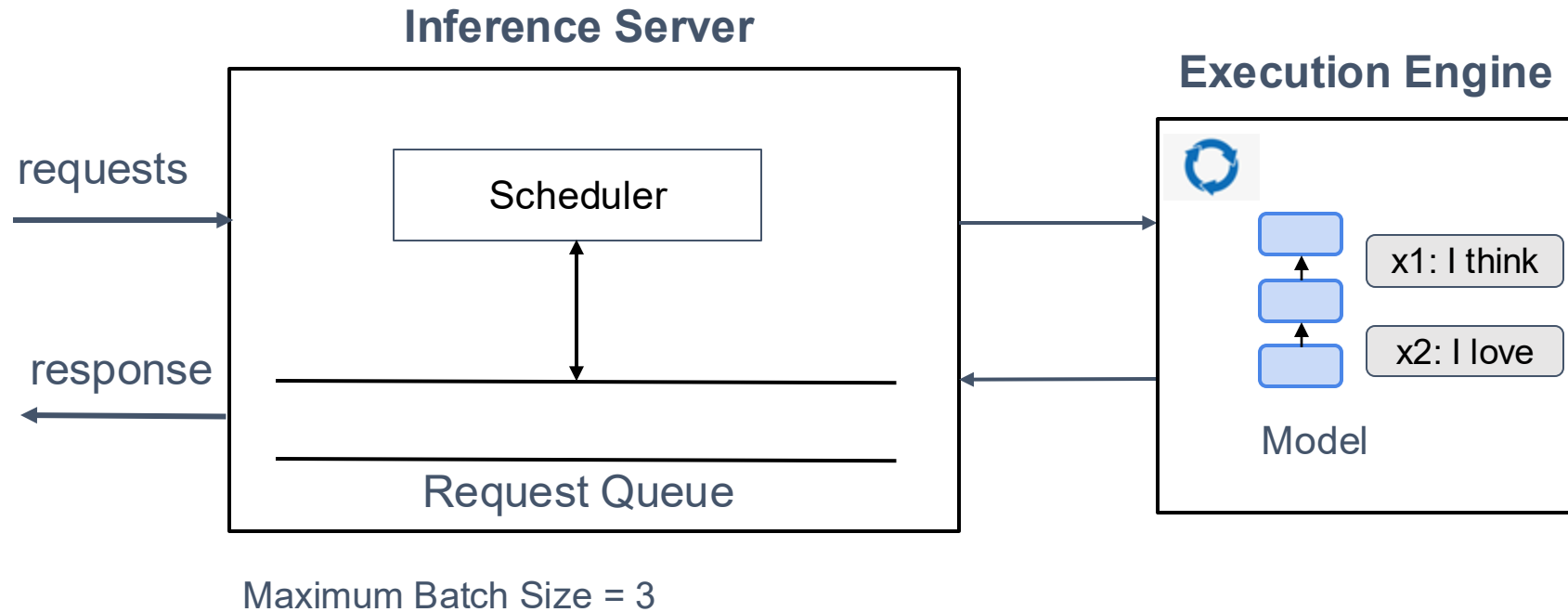
- **Goal:** Minimize TTFT, maximize throughput, and reduce TPOT
- **Throughput vs. TPOP Tradeoff:** Processing multiple queries concurrently increases throughput extends TPOT for each user.

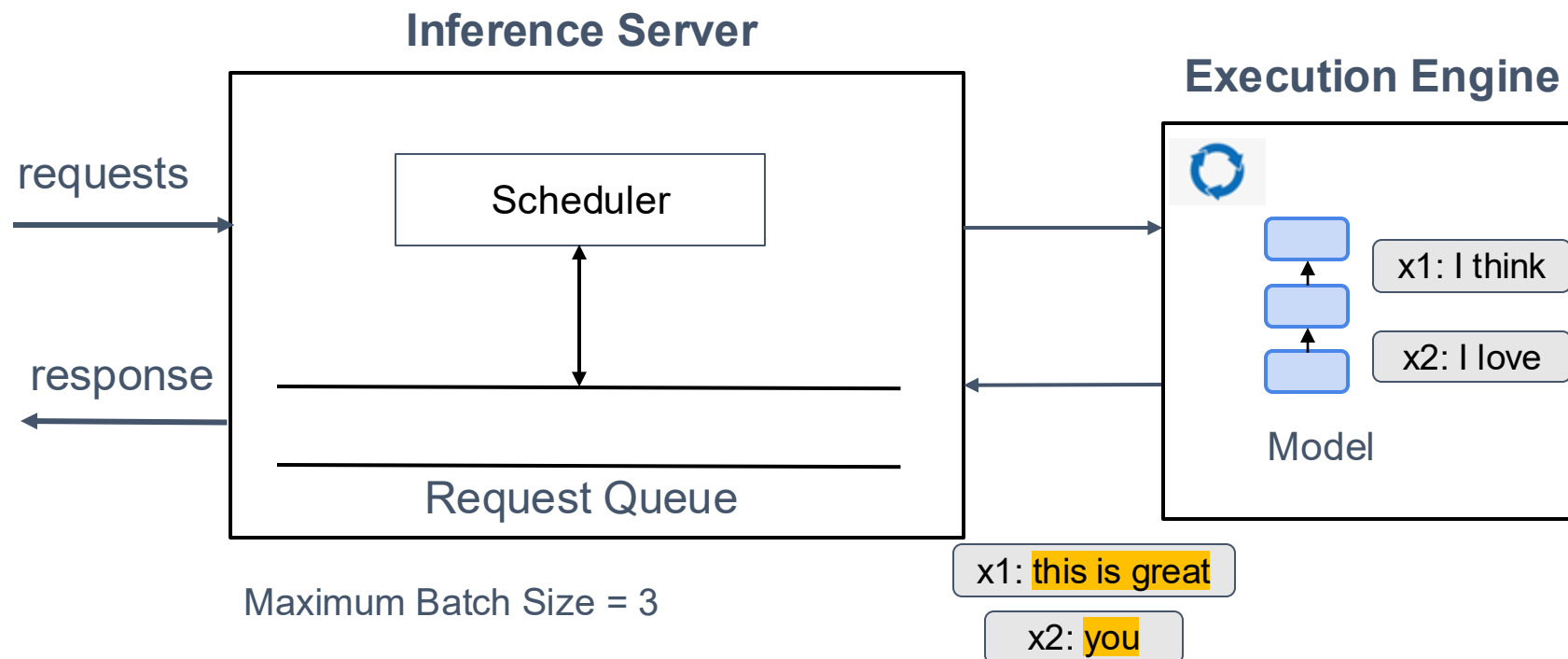
LLM Inference

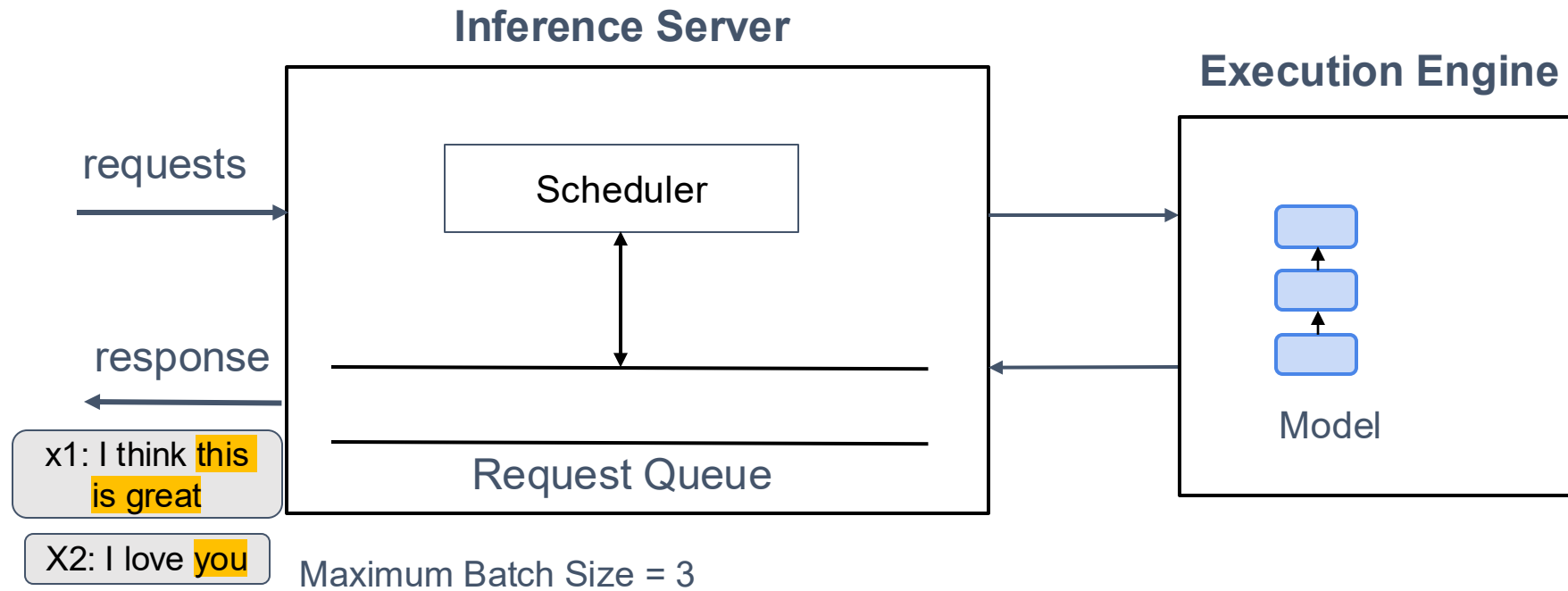
- Basics (Prefilling and Decoding)
- Continuous Batching
- Chunked-Prefill







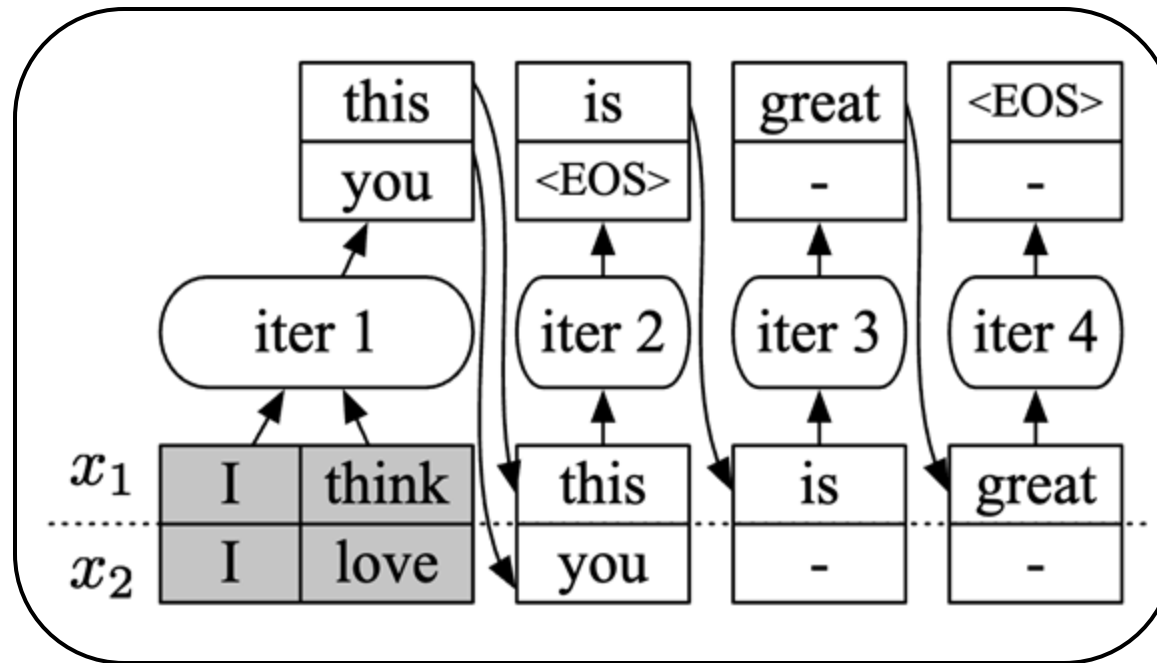




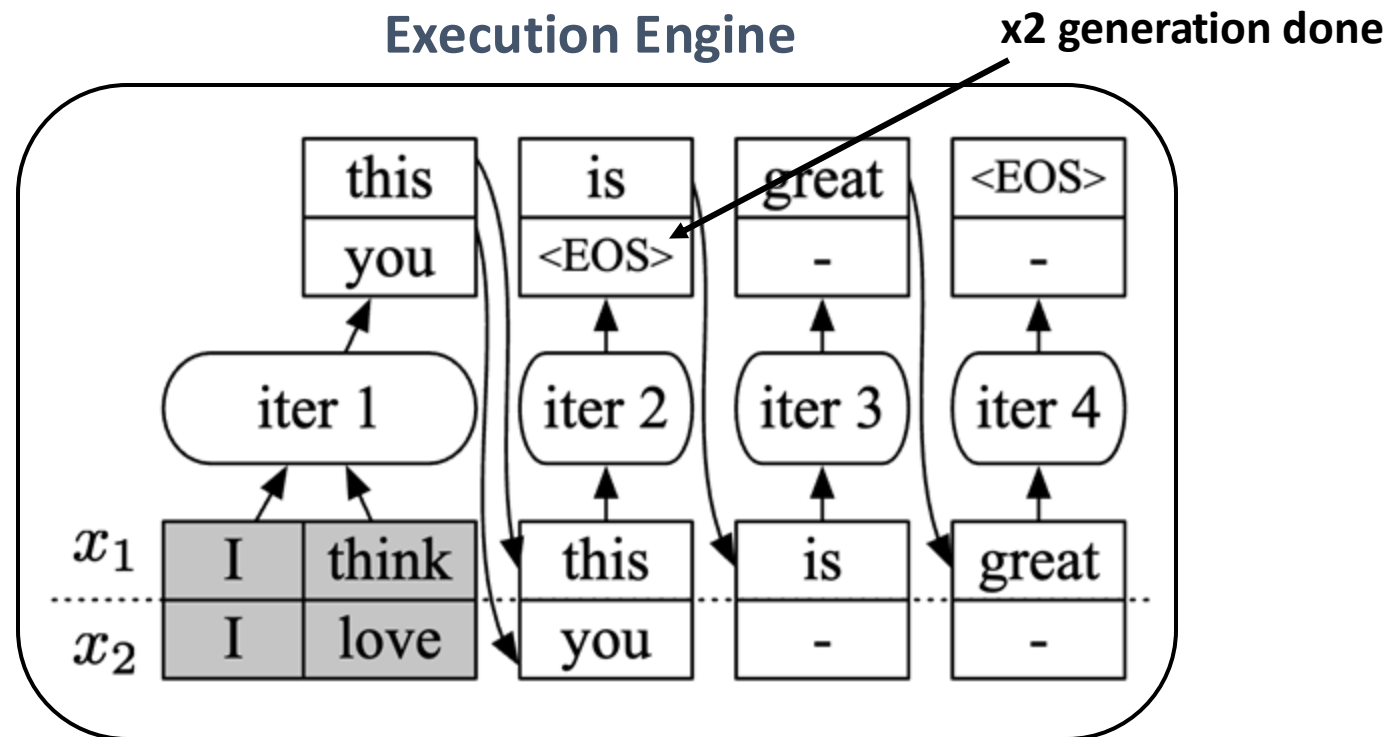
Problem 1: Request Level Scheduling



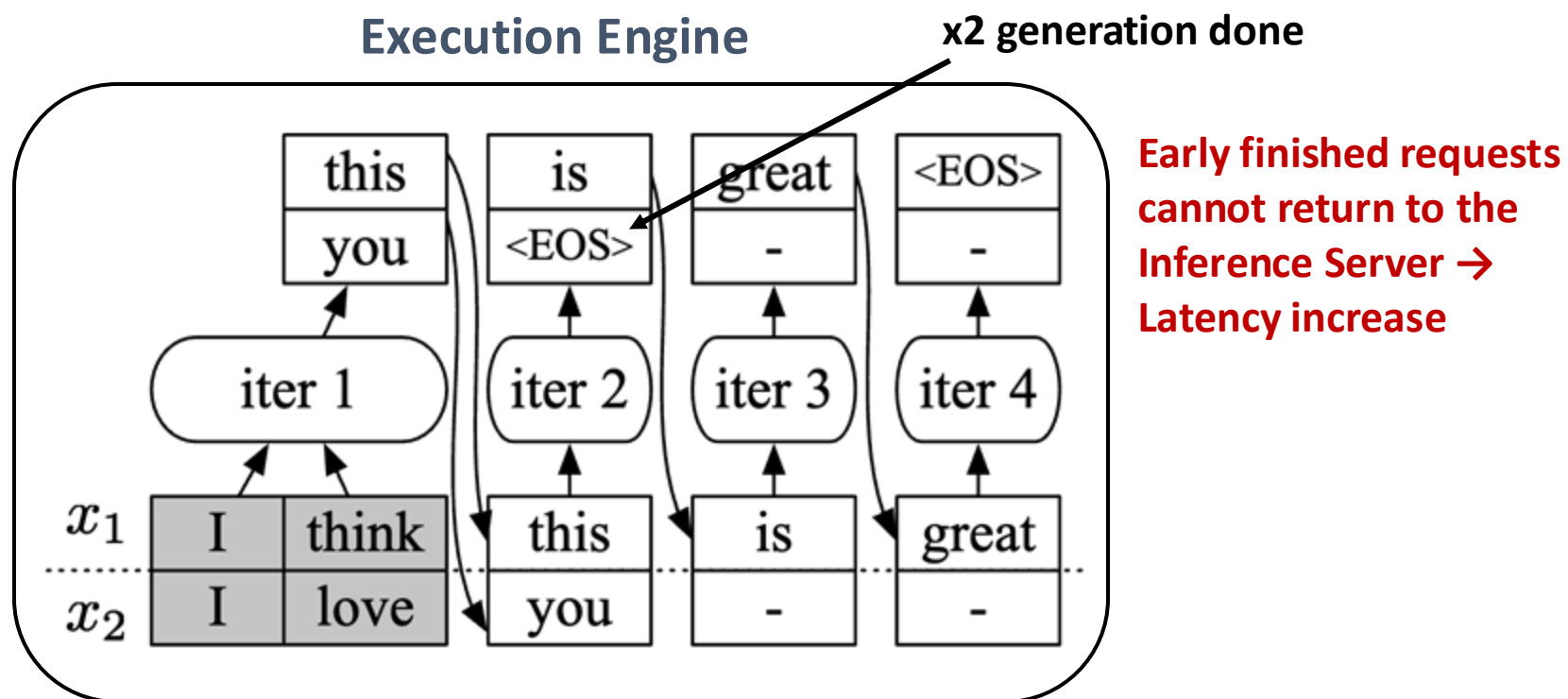
Execution Engine



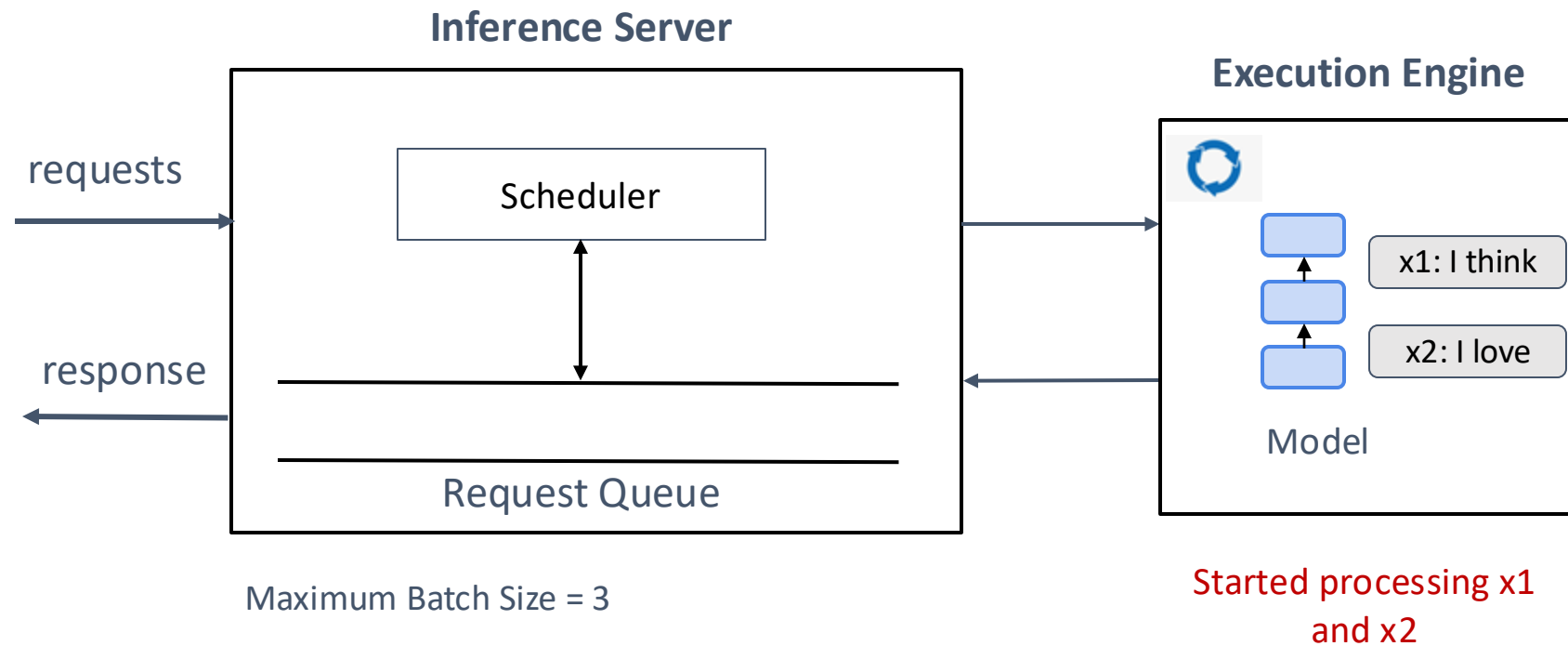
Problem 1: Request Level Scheduling



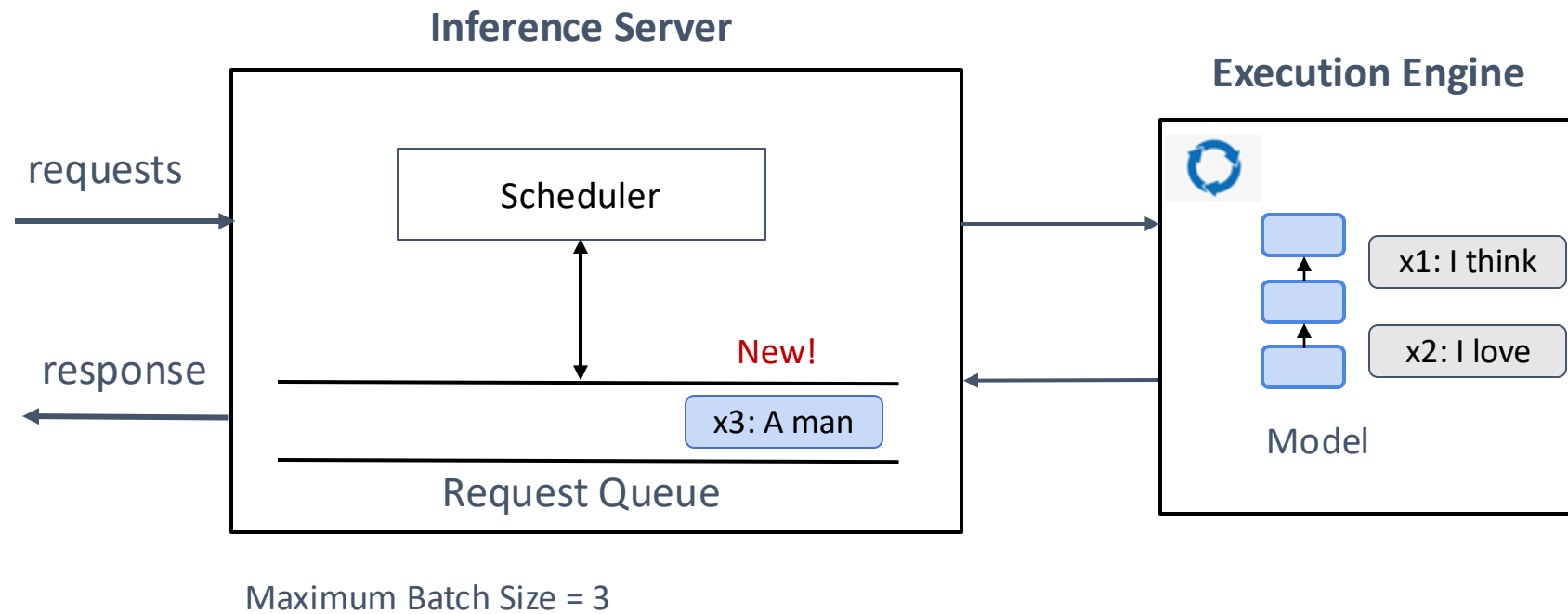
Problem 1: Request Level Scheduling



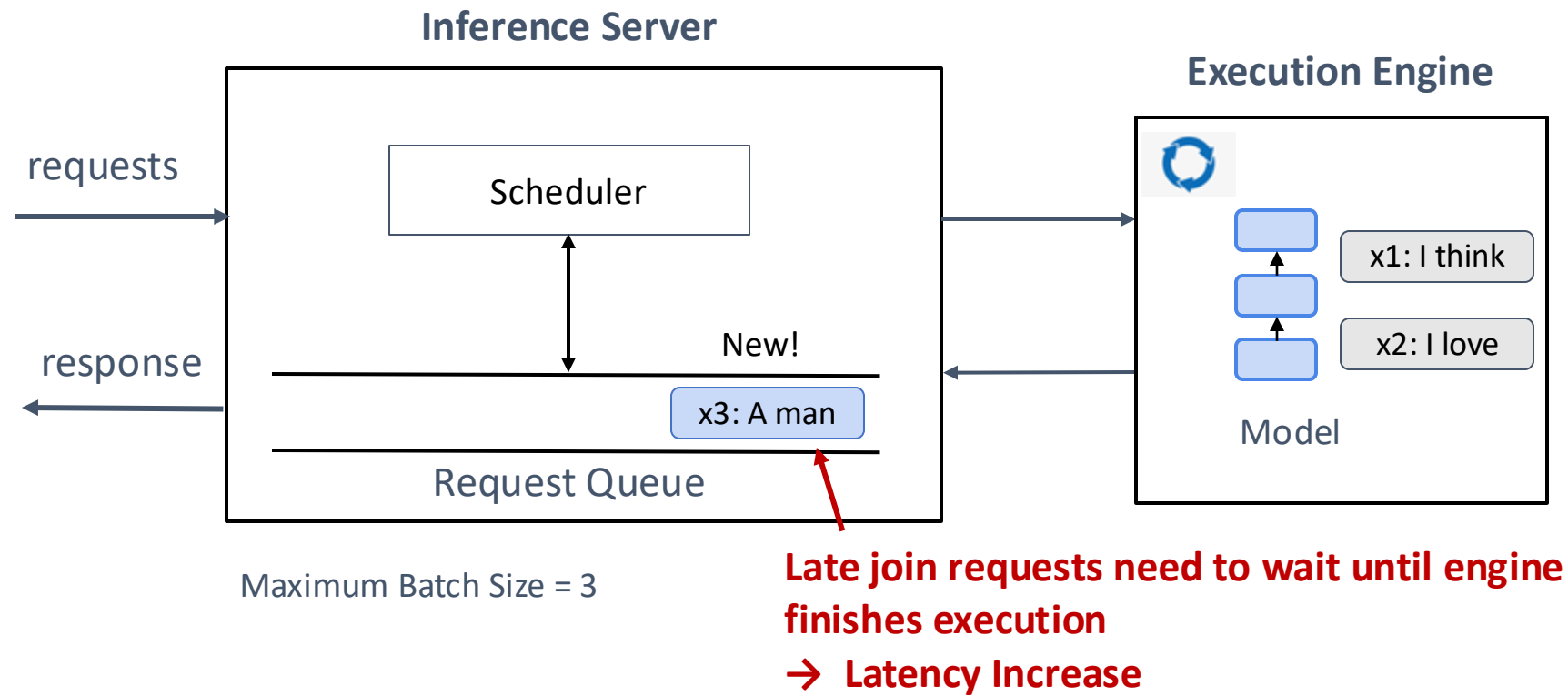
Problem 1: Request Level Scheduling



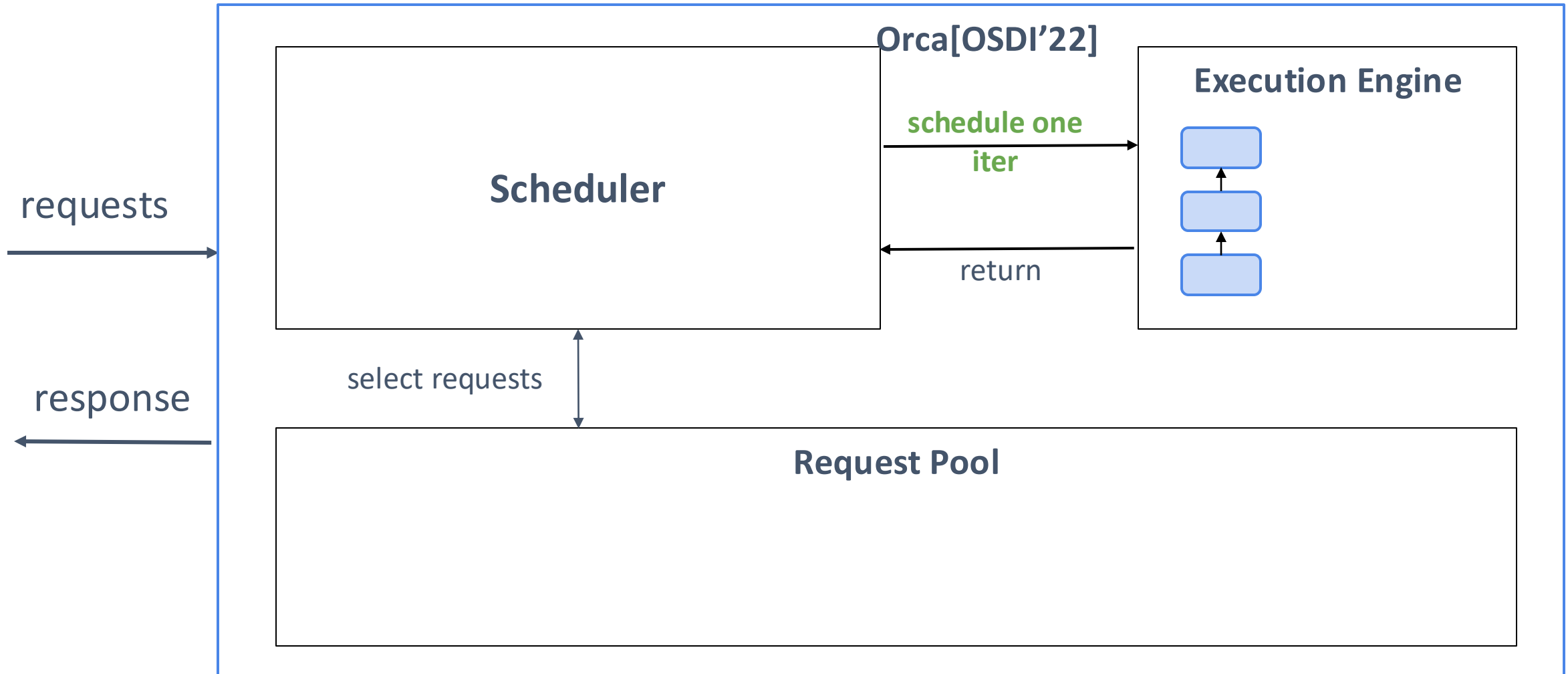
Problem 1: Request Level Scheduling



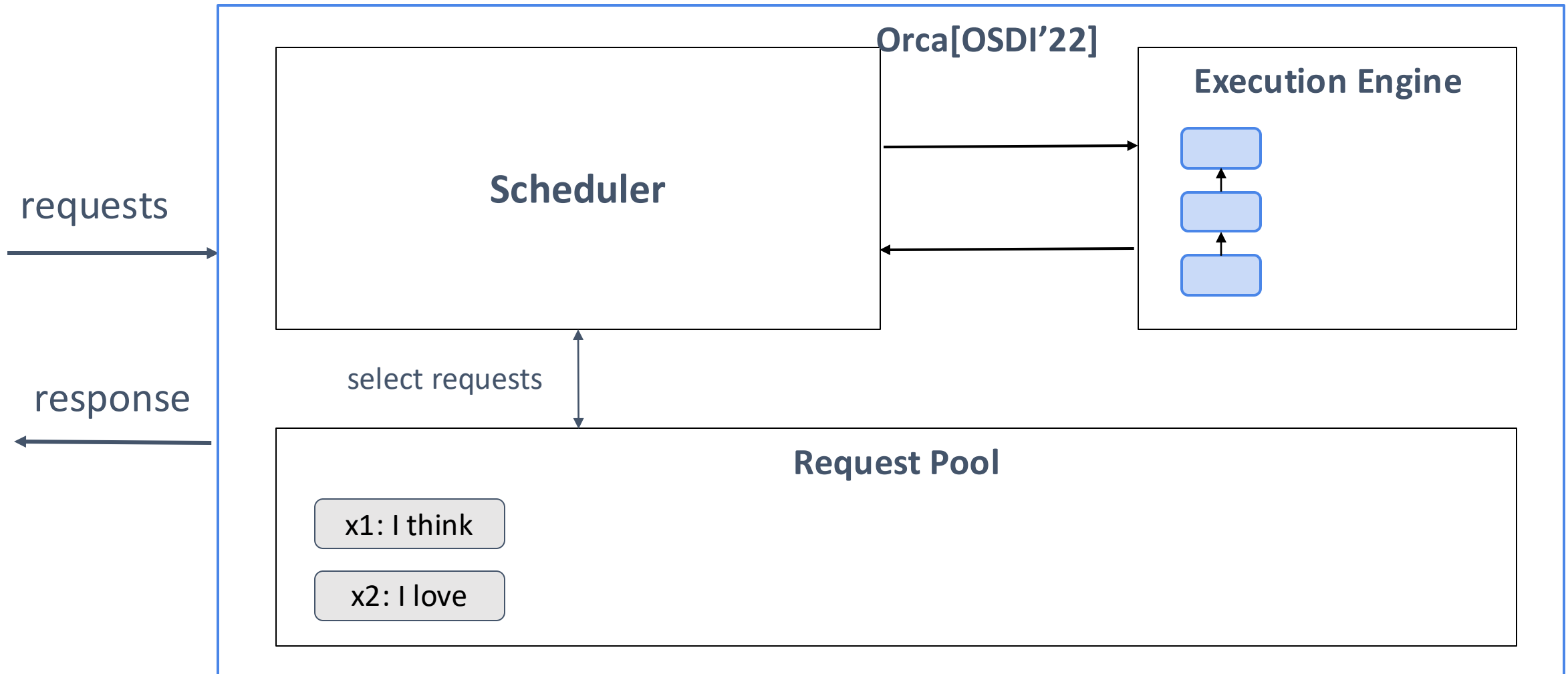
Problem 1: Request Level Scheduling



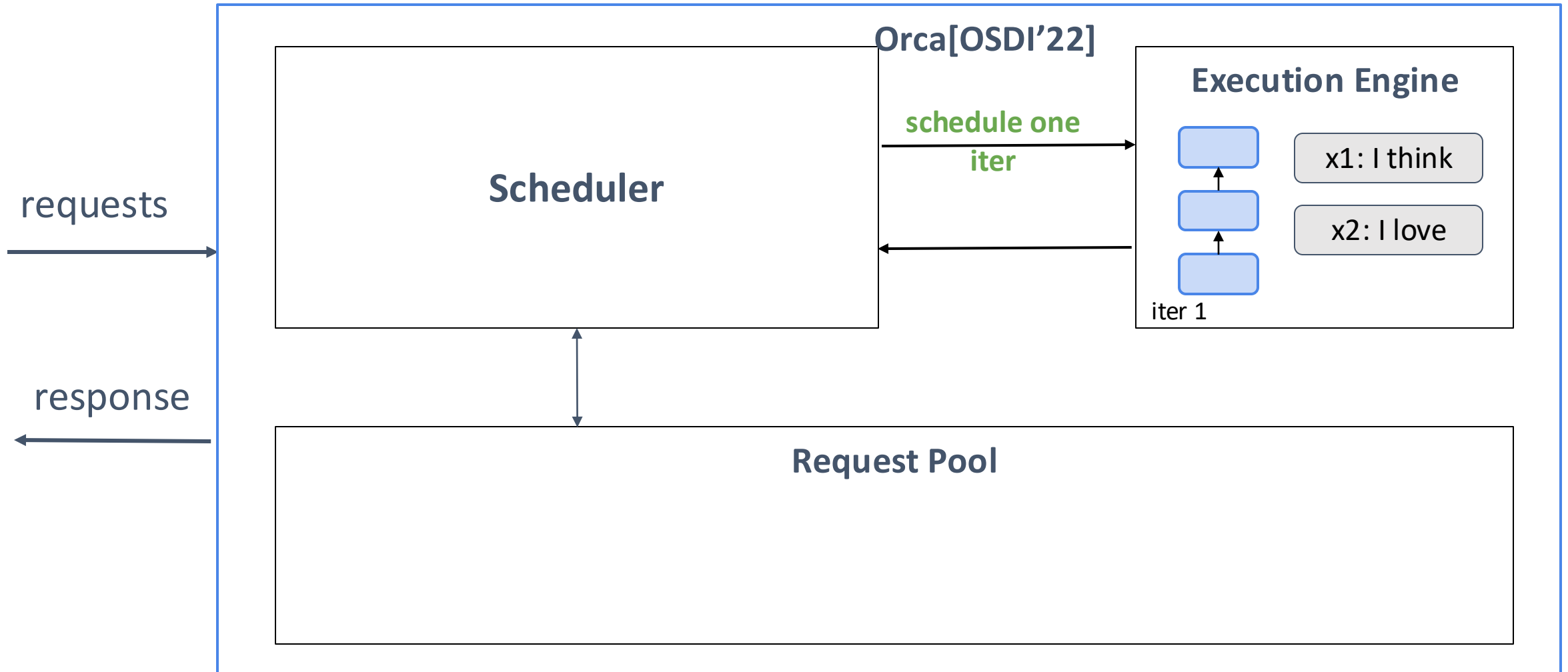
Solution 1: Iteration Level Scheduling



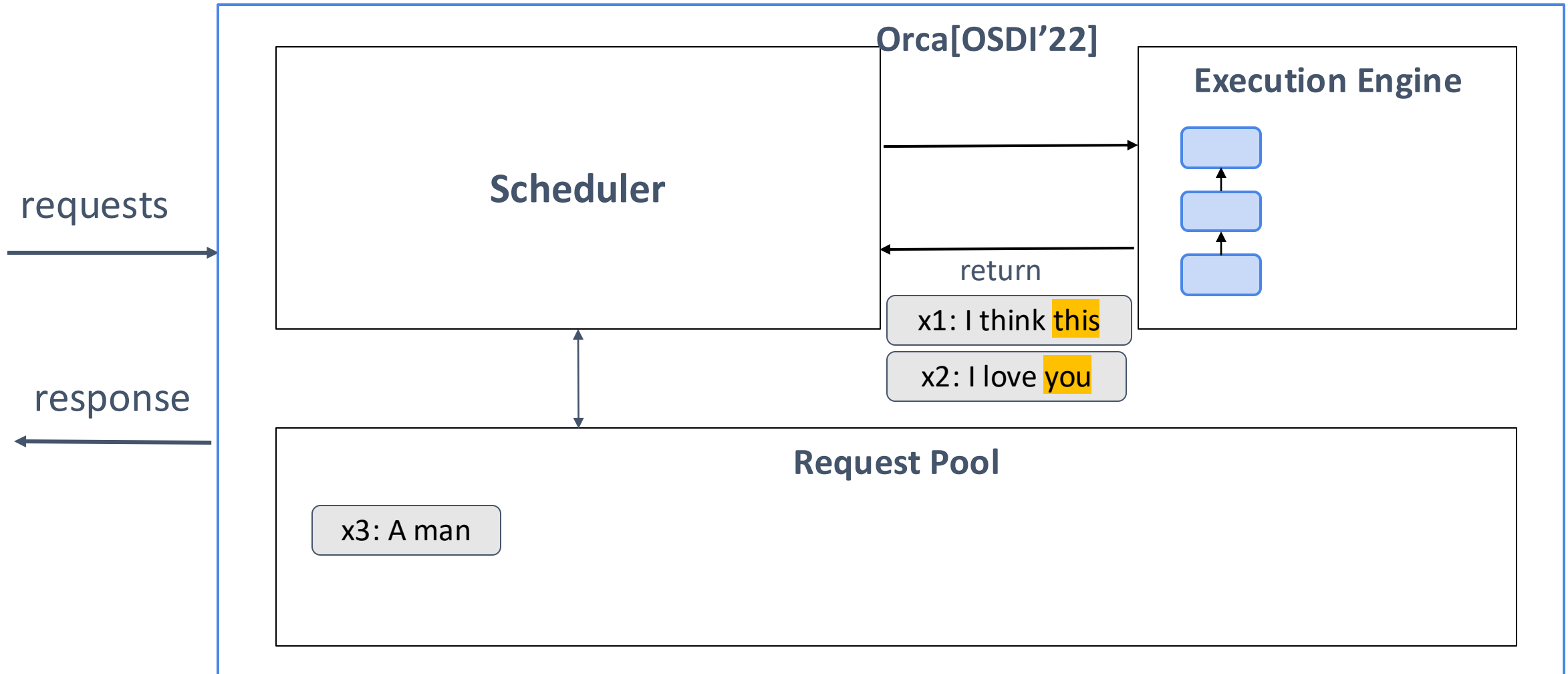
Solution 1: Iteration Level Scheduling



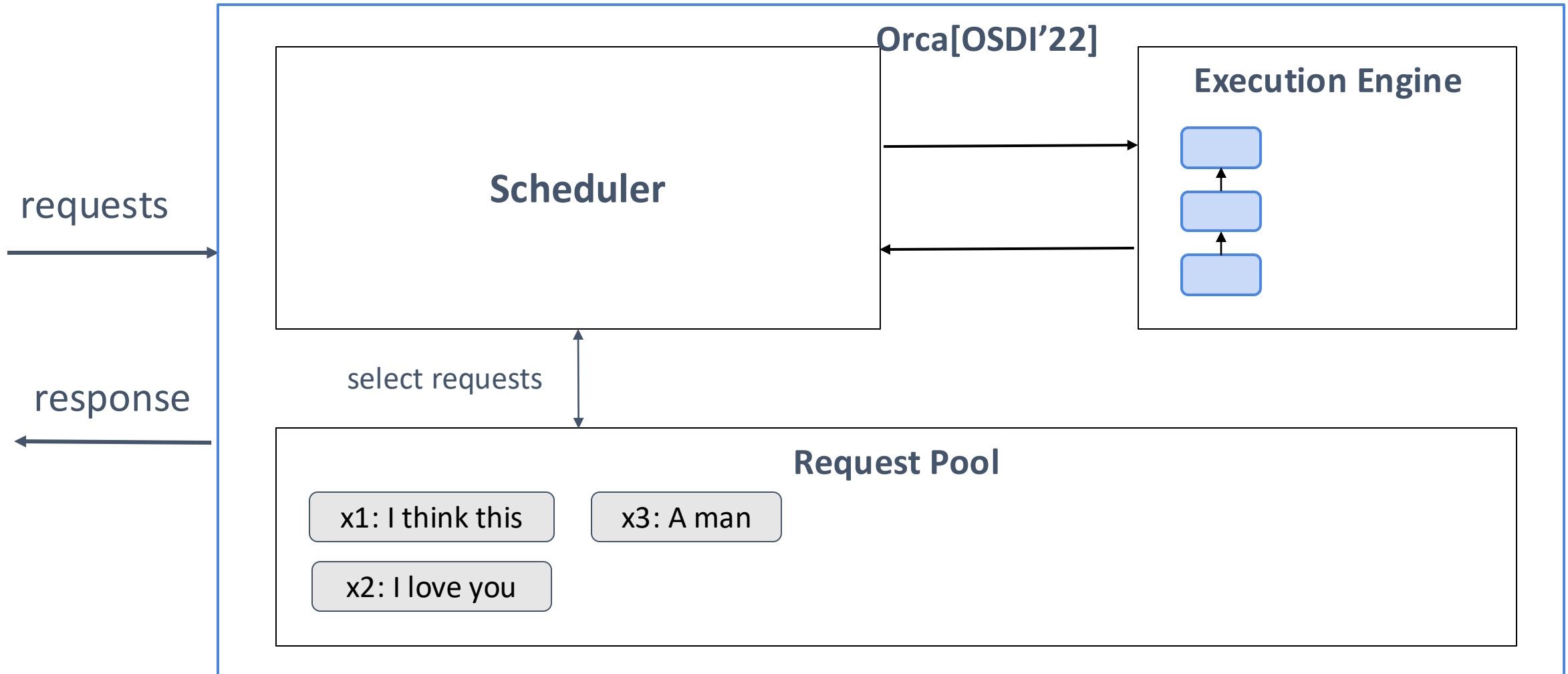
Solution 1: Iteration Level Scheduling



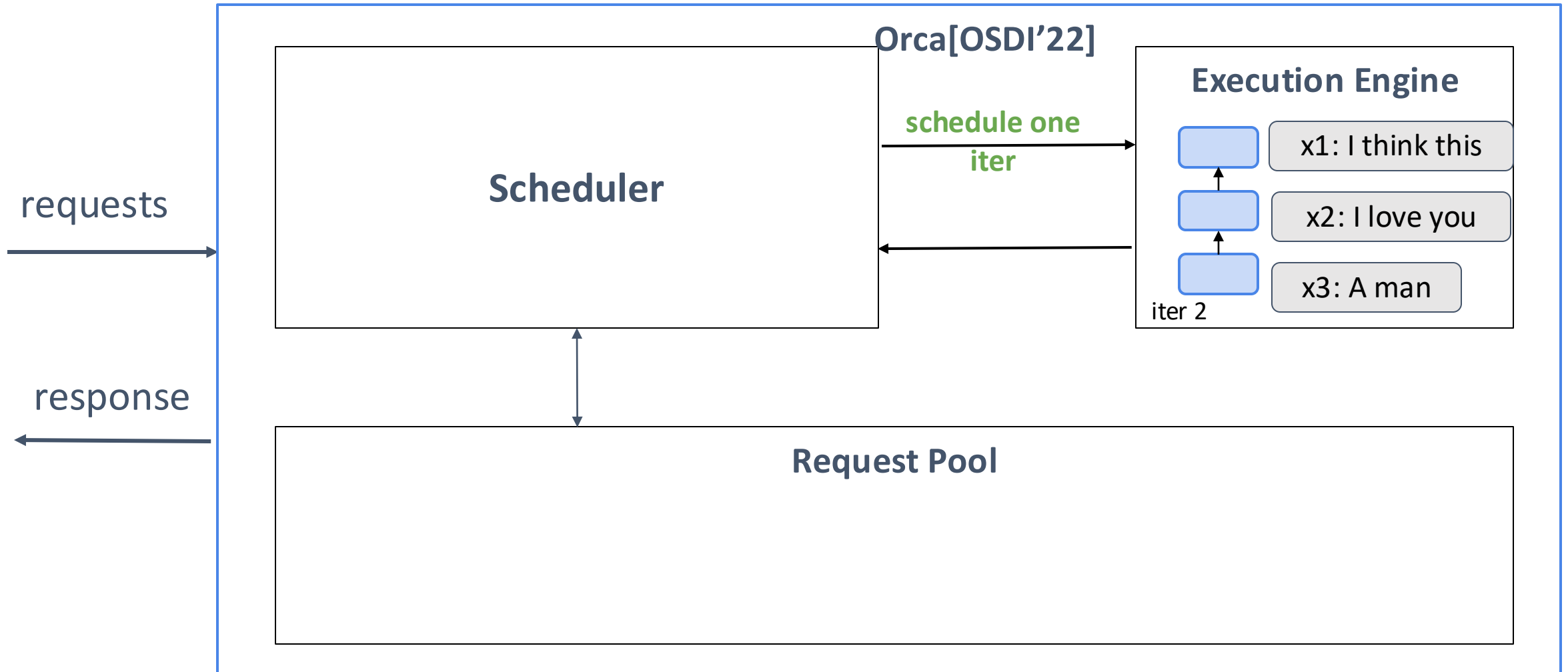
Solution 1: Iteration Level Scheduling



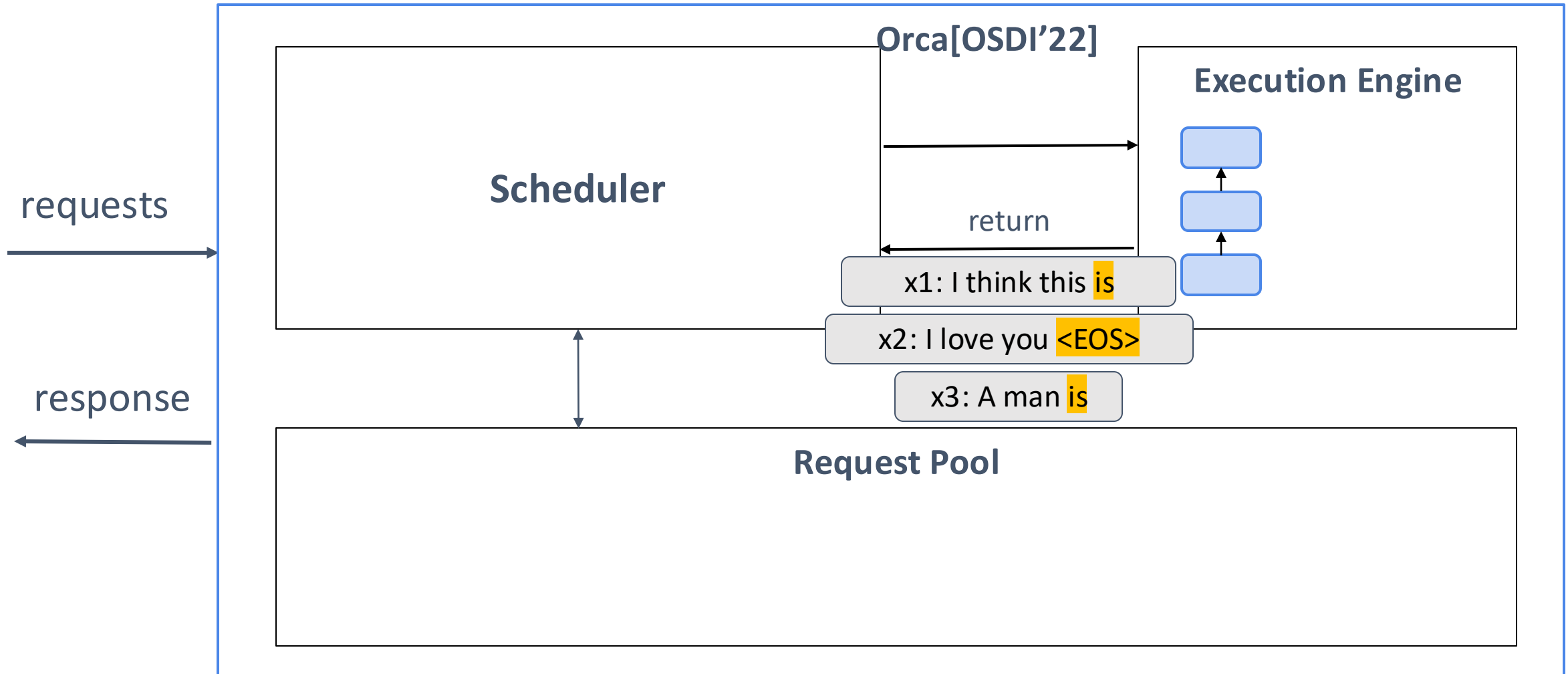
Solution 1: Iteration Level Scheduling



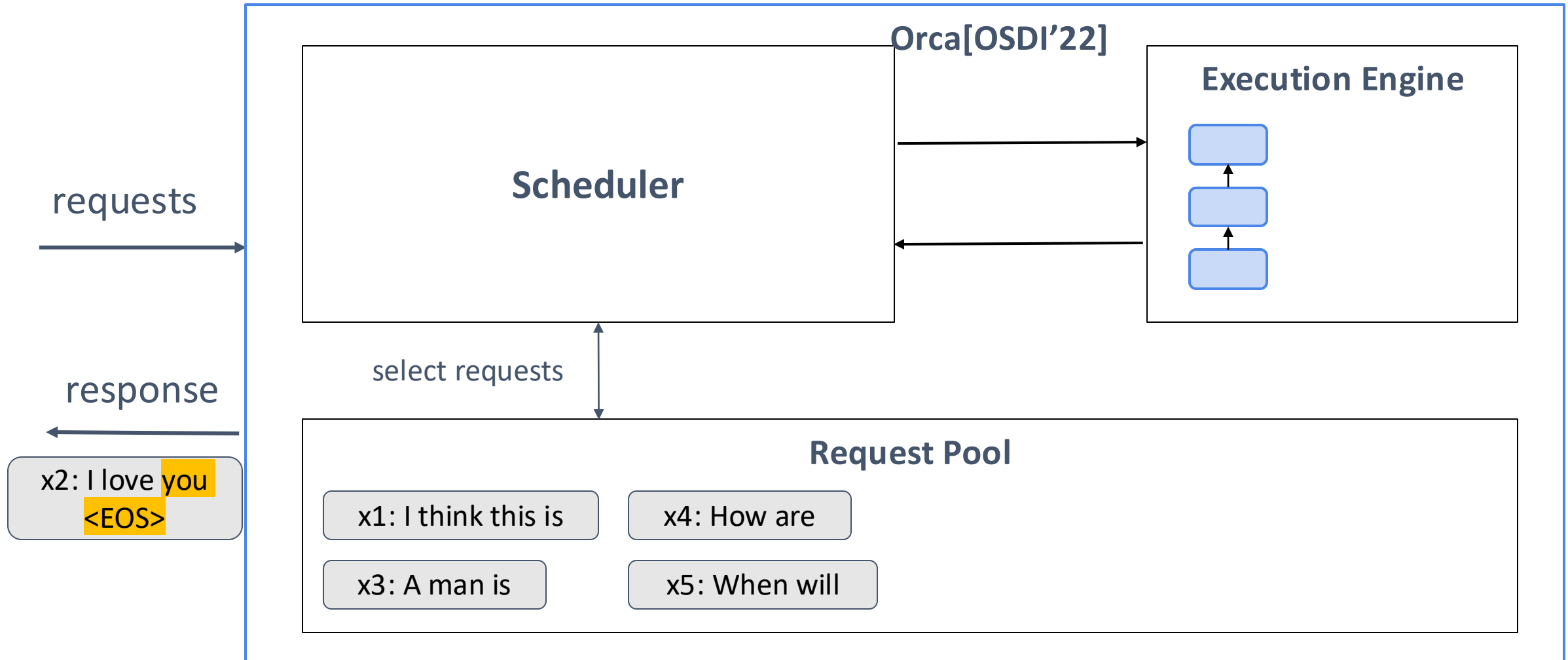
Solution 1: Iteration Level Scheduling



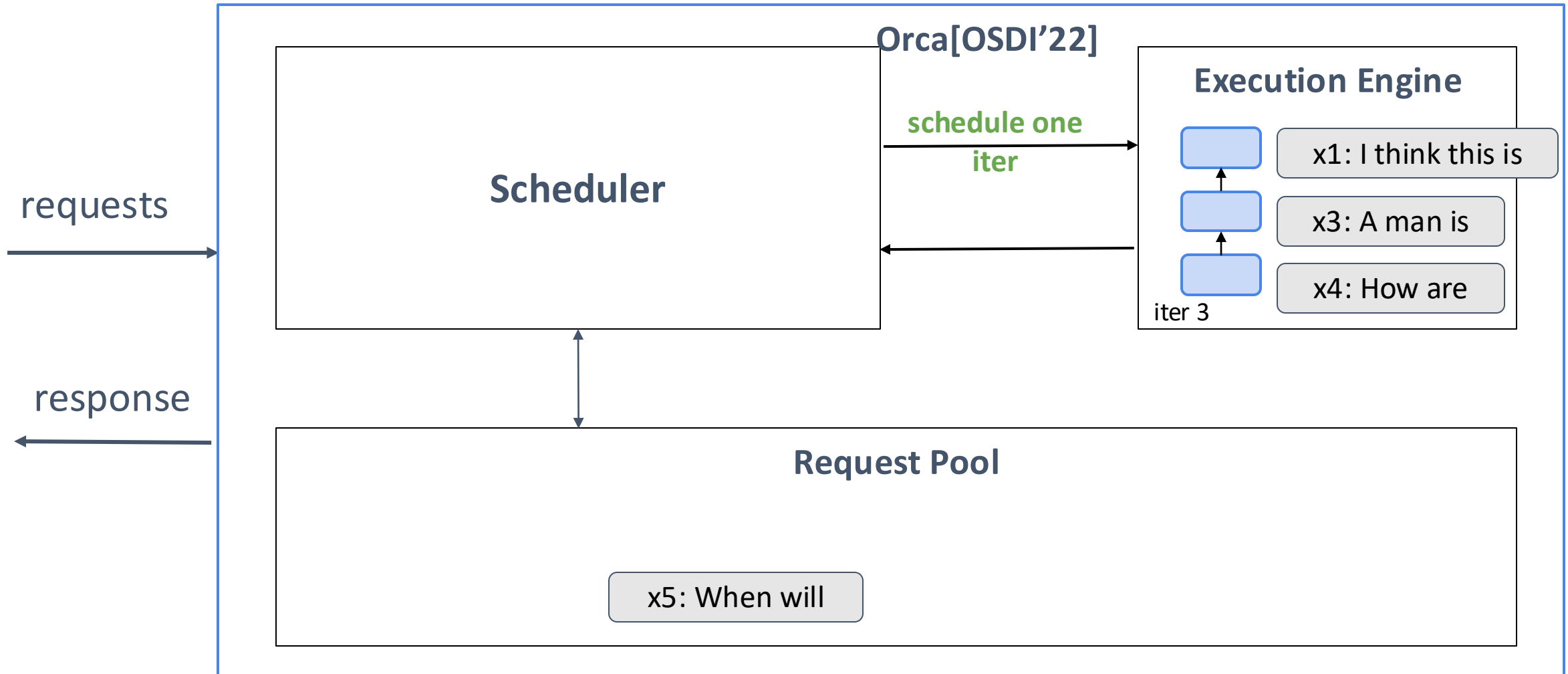
Solution 1: Iteration Level Scheduling



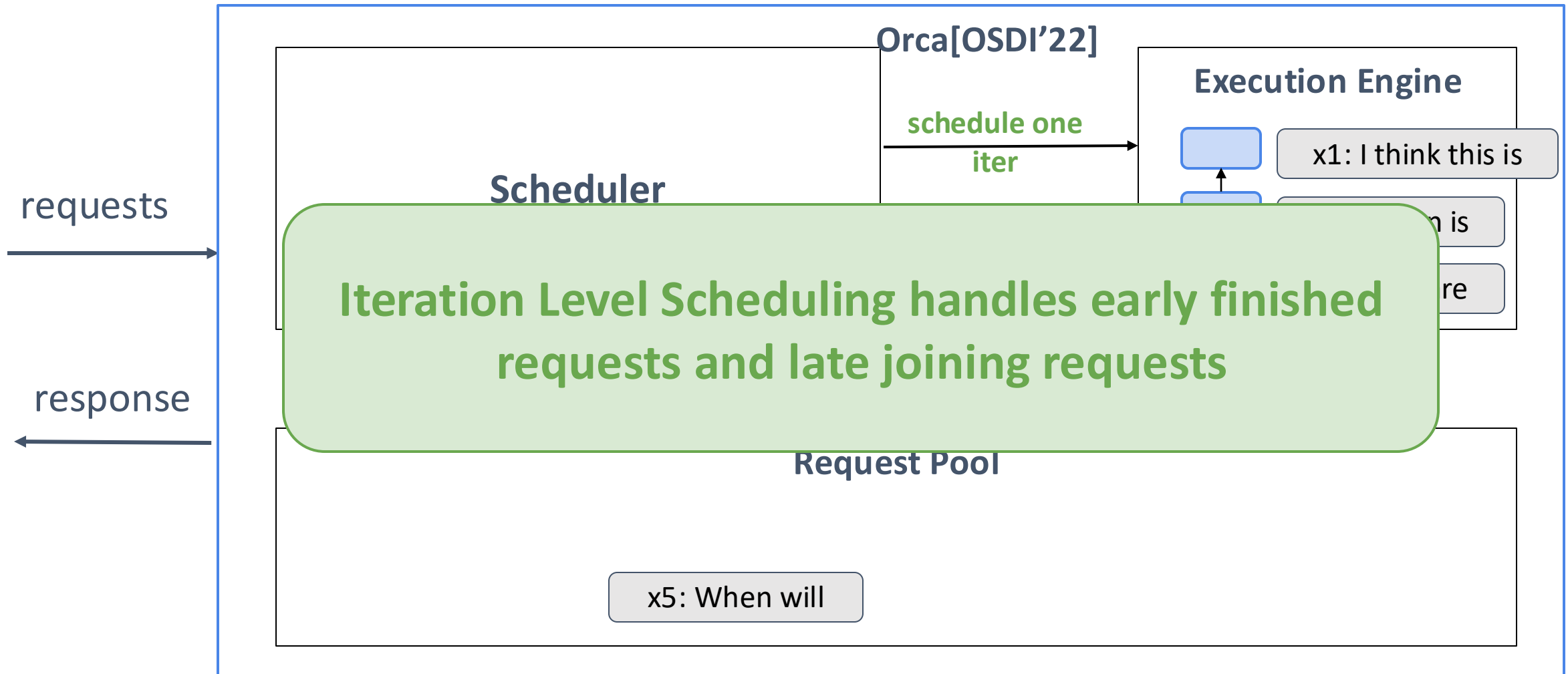
Solution 1: Iteration Level Scheduling



Solution 1: Iteration Level Scheduling



Solution 1: Iteration Level Scheduling

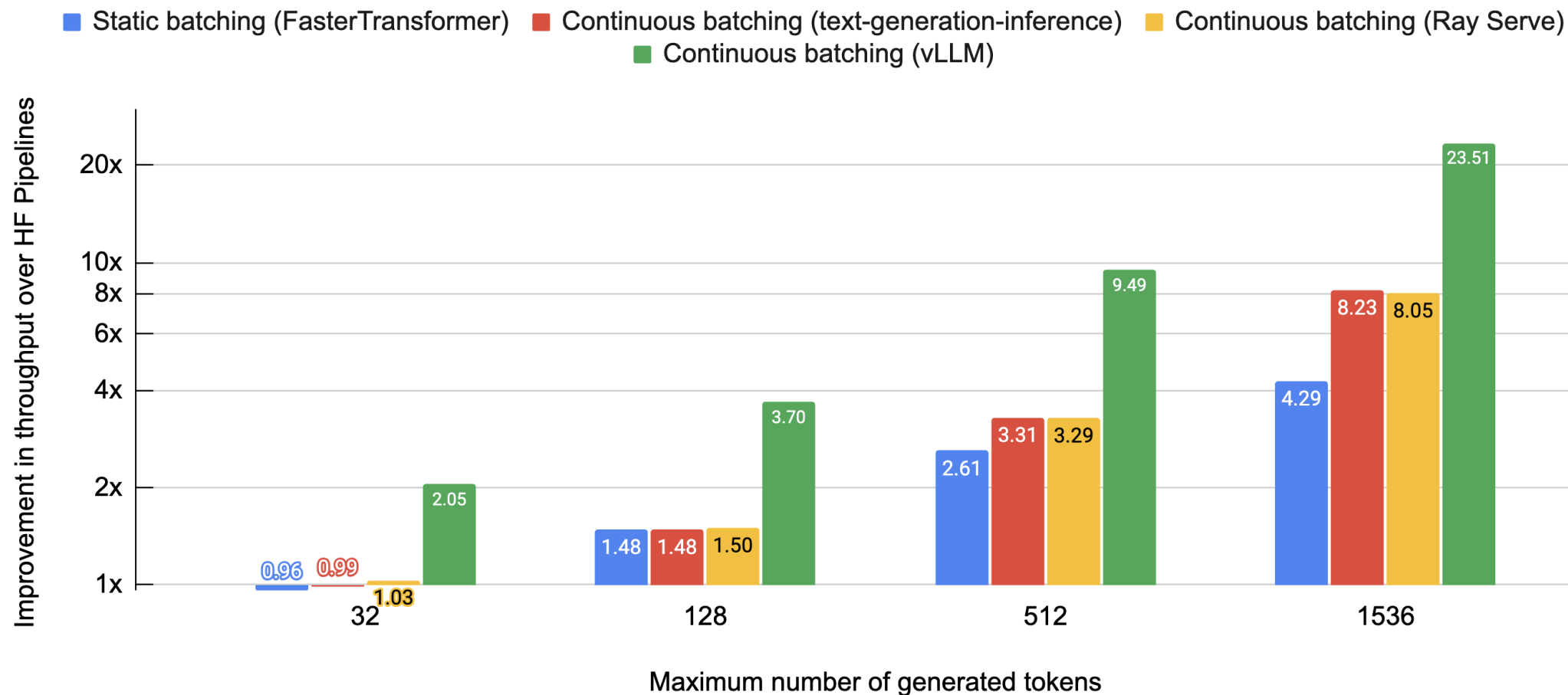


- Enforces iteration-level **first-come-first-served (FCFS)** property
- Maximum batch size → Throughput vs. Latency control knob
- Keep track of number of reserved slots to avoid deadlock
 - Slot := memory required for storing an Attention key and value for a single token

Throughput Improvement from Continuous Batching



Throughput improvement over naive static batching vs. generated sequence length variance



- Continuous batching handles early-finished and late-arrived requests more efficiently
- Fills GPU capacity after each token generation
- As variance in sequence length increases, continuous batching increases GPU utilization

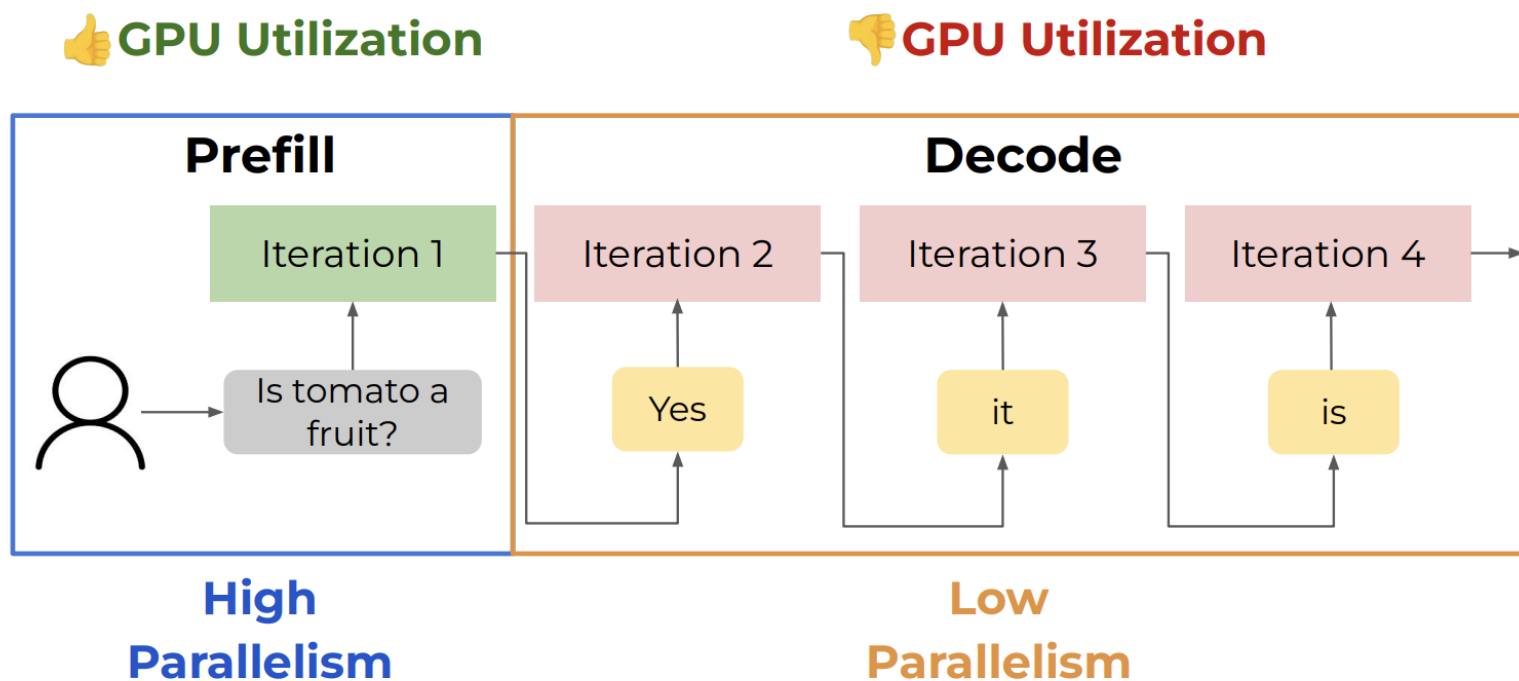
LLM Inference

- Basics (Prefilling and Decoding)
- Continuous Batching
- Chunked-Prefill

Chunked-Prefill in One Slide



Core idea: partition prefilling input into multiple chunks (e.g., 256 tokens/chunk) and mix with decoding requests

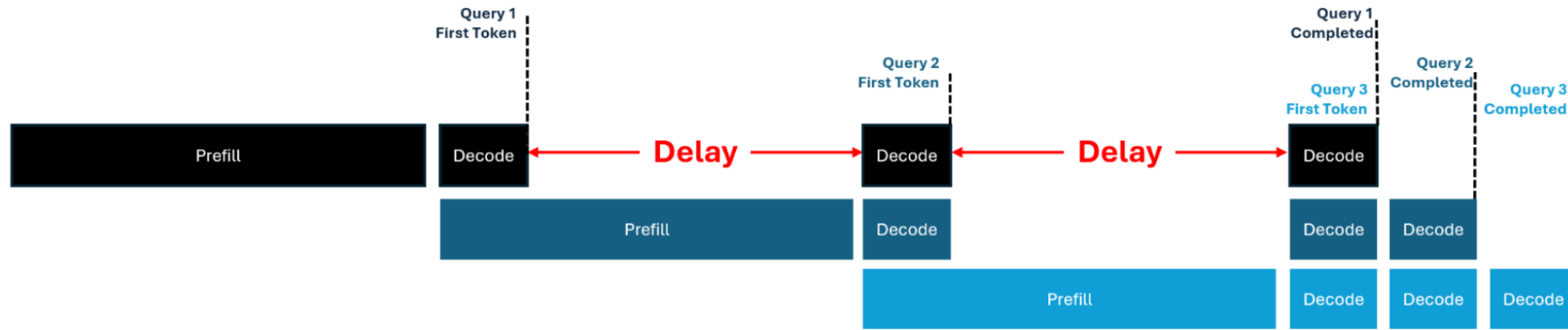


How about using a larger batch size?

Chunked-Prefill in One Slide



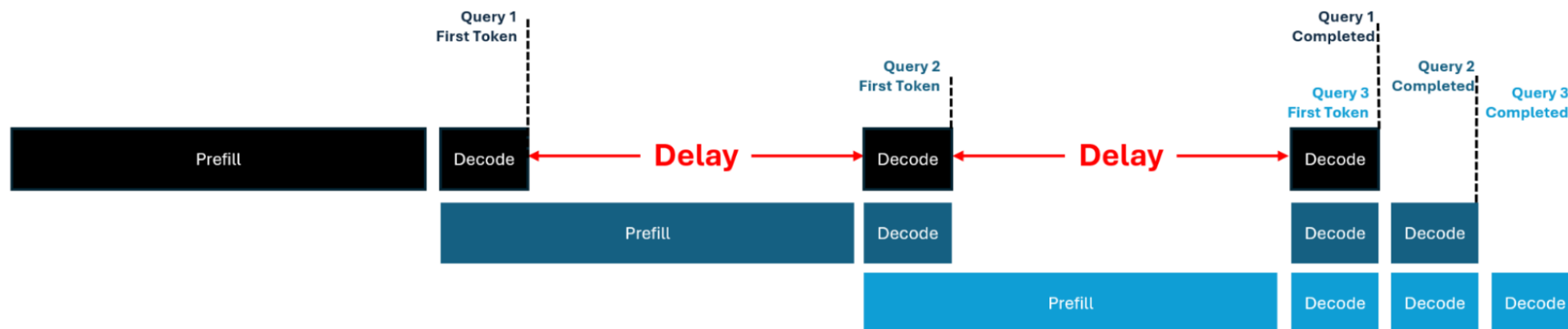
W/O TensorRT-LLM Chunked Prefill



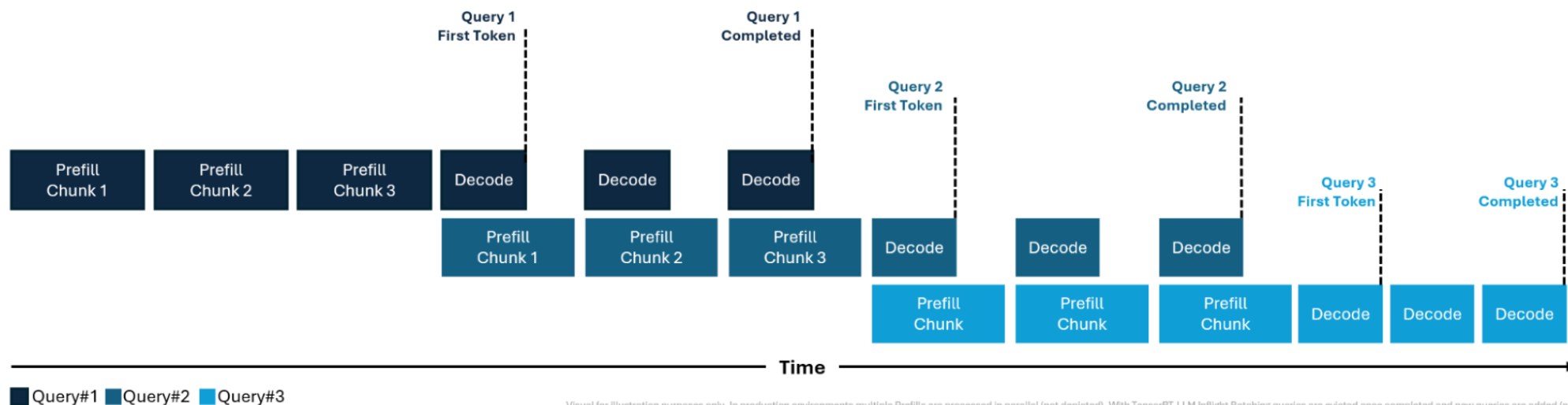
Chunked-Prefill in One Slide



W/O TensorRT-LLM Chunked Prefill



W/ TensorRT-LLM Chunked Prefill



Visual for illustration purposes only. In production environments multiple Prefills are processed in parallel (not depicted). With TensorRT-LLM Inflight Batching queries are evicted once completed and new queries are added (not depicted).

DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving

Yinmin Zhong¹ Shengyu Liu¹ Junda Chen³ Jianbo Hu¹ Yibo Zhu² Xuanzhe Liu¹
Xin Jin¹ Hao Zhang³

¹School of Computer Science, Peking University ²StepFun ³UC San Diego

Use separate GPU instances for prefilling and decoding stages

- Prefill Instances: process prefilling requests only
- Decode Instances: process a larger batch of decoding requests

Questions?

Problem 2: How to Batch Requests?



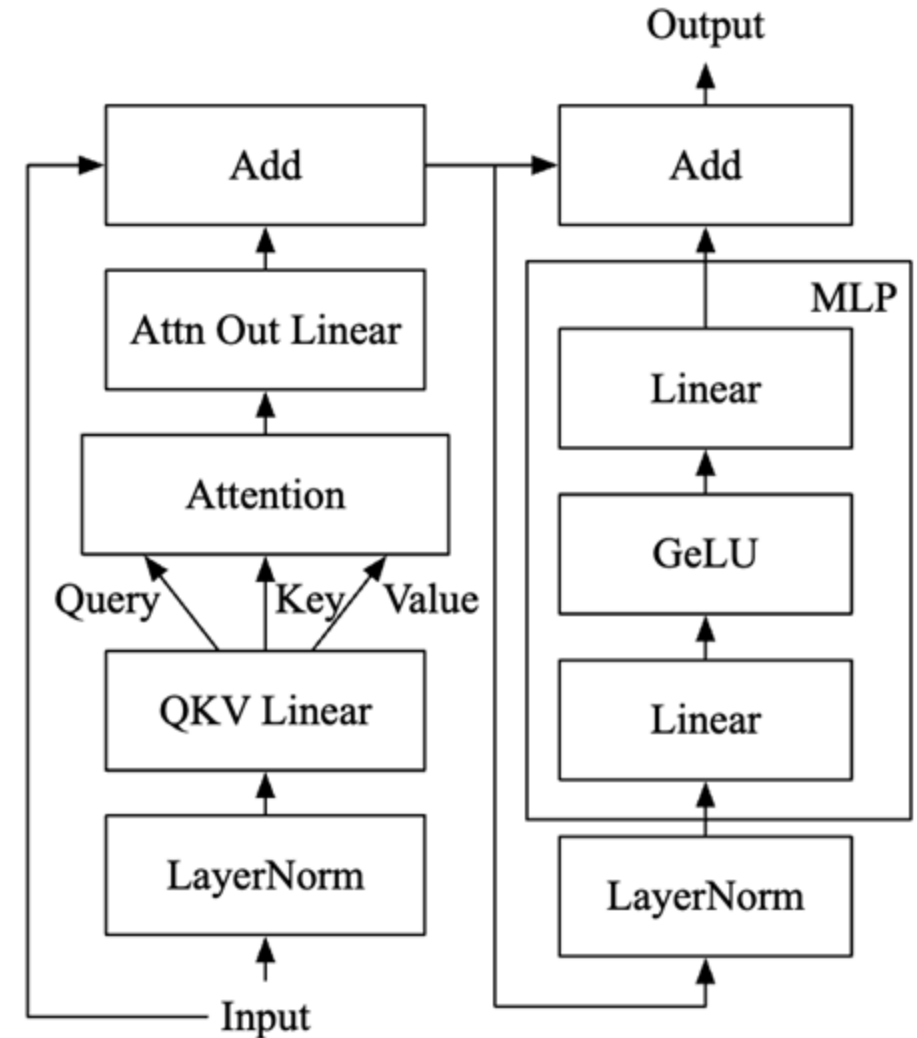
Let's assume Batch Size $B = 1$

Input Dimension: $[L \times H]$ (L=sequence length, H=hidden dim.)

Attention Operation:

1. $QK^T : [L \times H] \times [H \times L] \rightarrow [L \times L]$
2. $P = \text{softmax}(QK^T) : [L \times L]$
3. $O = PV : [L \times L] \times [L \times H] \rightarrow [L \times H]$

With Batch Size B , QK^T will be $[B \times L \times L]$



Problem 2: How to Batch Requests?



Let's assume Batch Size $B = 1$

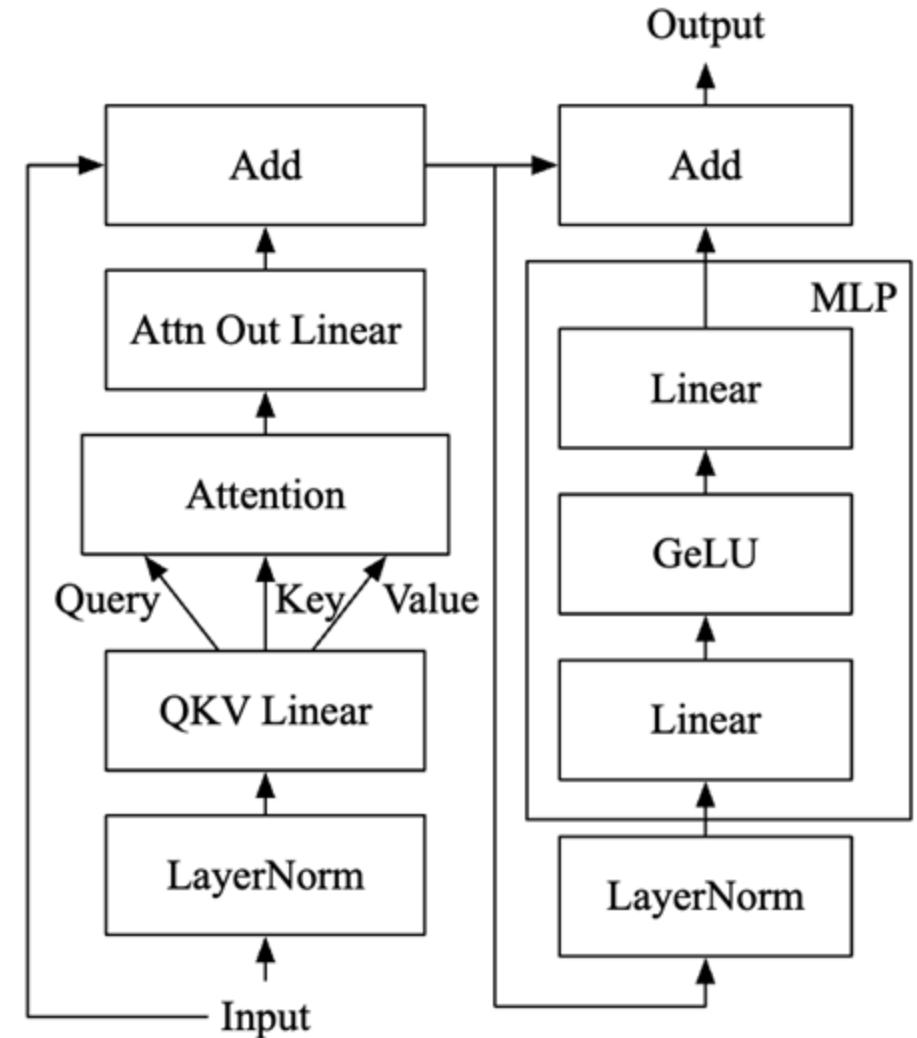
Input Dimension: $[L \times H]$ (L =sequence length, H =hidden dim.)

Attention Operation:

1. $QK^T : [L \times H] \times [H \times L] \rightarrow [L \times L]$
2. $P = \text{softmax}(QK^T) : [L \times L]$
3. $O = PV : [L \times L] \times [L \times H] \rightarrow [L \times H]$

With Batch Size B , QK^T will be $[B \times L \times L]$

With different sequence lengths, QK^T
cannot be computed



Solution 2: Selective Batching



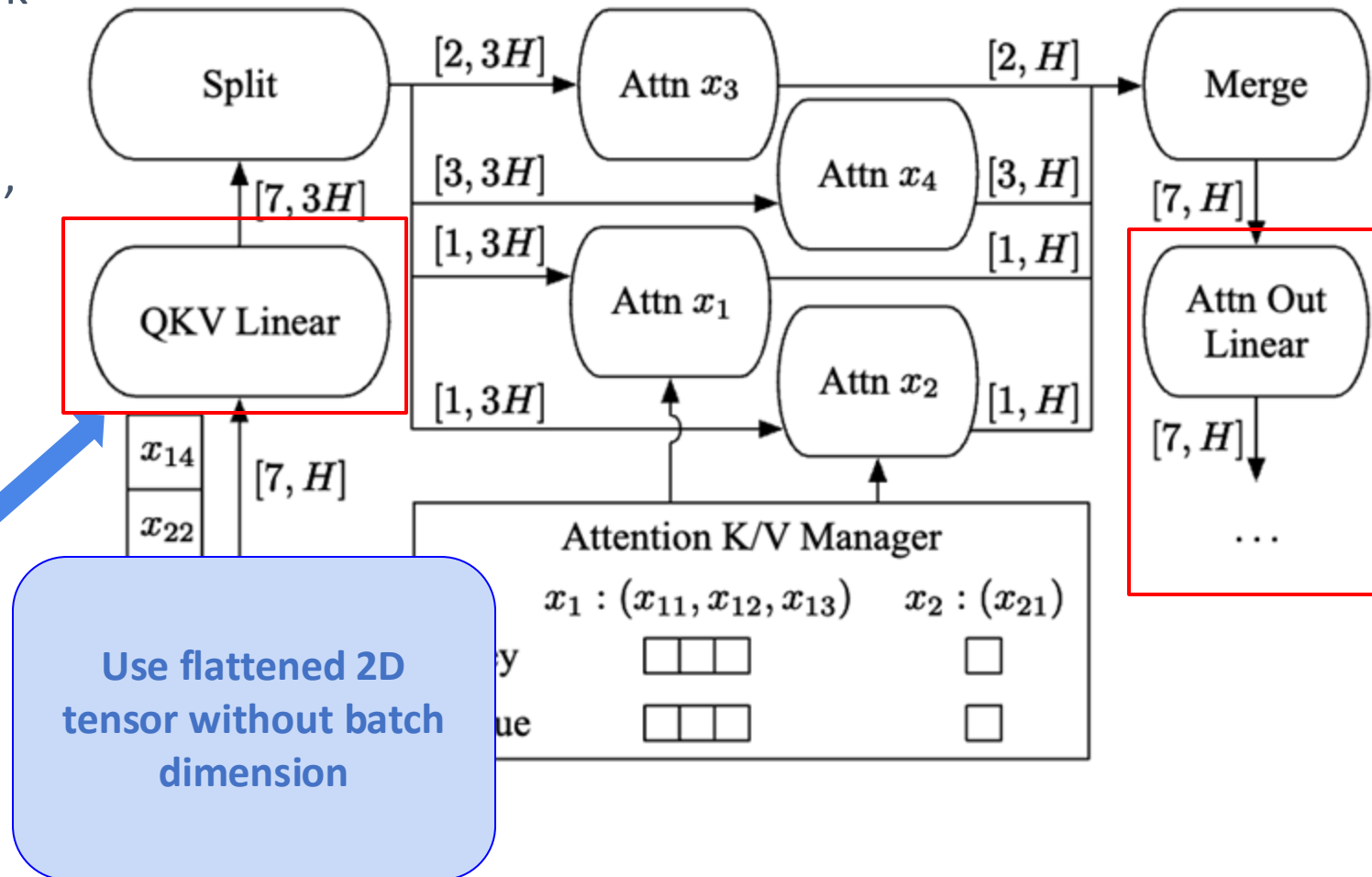
Only **Attention operation** does not work with batching tensors with diff. L_i

Batch for other ops. (Layer Norm, GeLU, etc.)

Coalesce $[L_i, H]$ tensor to $[\sum L_i, H]$ for batching

x1: [1,H]
x2: [1,H]
x3: [2,H]
x4: [3,H]

→ **[7,H]
tensor**



Solution 2: Selective Batching

I

Only **Attention operation** does not work with batching tensors with diff. L_i

Batch for other ops. (Layer Norm, GeLU, etc.)

Coalesce $[L_i, H]$ tensor to $[\sum L_i, H]$ for batching

$x_1: [1, H]$

$x_2: [1, H]$

$x_3: [2, H]$

$x_4: [3, H]$



**$[7, H]$
tensor**

